Victoree Norris
Wassil Mian

>_ 01 · EXECUTIVE SUMMARY

Credit risk model: Amex default prediction

Machine Learning Project

pitch.md

GOAL	Predict which card applicants will default, then decide who gets approved.
WHY IT MATTERS	A default erases a year of interest and spend revenue. AI scoring ranks that risk before we lend.
HOW IT HELPS	It approves the most revenue while keeping the book under the firm's 10% default limit.

RECOMMENDED STRATEGY

Conservative

Threshold **0.32** · XGBoost model
Default rate **4.87%** train

Aggressive (0.54) reaches 9.86% default rate

table_01 · Strategy results by sample (DR = default rate · revenue in feature units)

		Train			Test 1			Test 2		
Strategy	Thr.	#Total	DR	Revenue	#Total	DR	Revenue	#Total	DR	Revenue
Conservative	0.32	42,729	4.87%	541.72	9,159	5.80%	115.12	9,161	5.95%	114.28
Aggressive	0.54	49,207	9.86%	766.53	10,566	10.90%	162.03	10,584	11.00%	161.14

Data: up to 13 months of history per account

split · 64,248 / 13,767 / 13,768

dataset.md

SOURCE

Kaggle Amex default data. Our sample: 91,783 accounts, 25.65% default.

WINDOW

Each account is scored at its latest statement, March 2018 in this file. The six-month window runs Oct 2017 to Mar 2018.

TARGET

default = 1 when the due amount stays unpaid 120 days after the latest statement.

table_02 · Default rate by history length

History	Accounts	Default rate	History	Accounts	Default rate
All	91,783	25.65%	7 months	1,046	41.49%
13 months	77,348	22.94%	6 months	1,109	41.30%
12 months	2,115	37.87%	5 months	933	39.44%
11 months	1,159	44.09%	4 months	938	43.07%
10 months	1,329	46.58%	3 months	1,158	35.75%
9 months	1,278	43.58%	2 months	1,217	31.22%
8 months	1,169	45.08%	1 month	984	33.13%

DEVELOPMENT SAMPLE 20% of train_labels accounts; 10 of train_data features

>_ 03 · RAW FEATURES

Raw features: 17 fields in four families

src · development sample, before feature engineering

table_03 · Raw features by category

Category	# of features
Delinquency · D_*	10
Spend · S_*	1
Payment · P_*	0
Balance · B_*	4
Risk · R_*	2
Total	17

0 categorical | 17 numeric

glossary.md · What each family measures

D_* Delinquency
10 fields · late and missed payment signals

S_* Spend
1 field · S_27

P_* Payment
0 fields · none landed in this draw

B_* Balance
4 fields · B_12, B_28, B_32, B_37

R_* Risk
2 fields · R_13, R_28

TAKEAWAY Seventeen raw fields feed 374 summaries. 22 columns per feature

Feature engineering: 17 fields become 374 summaries

windows · 3, 6, 9, 12 months

table_04a · By category

	D_*	S_*	P_*	B_*	R_*	Total
Built	220	22	0	88	44	374
Kept	23	1	0	8	4	36

table_04b · By statistic type

	Ave	Sum	Min	Max	Med	Last	YoY
Built	68	68	68	68	68	17	17
Kept	9	2	4	7	8	4	2

table_04c · Top 5 features by importance · summary statistics (Test 2)

Feature	Min	1st	5th	Median	95th	99th	Max	Mean	Missing
D_48_Latest	-0.009	0.001	0.013	0.310	0.955	1.05	1.98	0.406	12.46%
B_37_Latest	-0.473	0.000	0.002	0.033	0.656	1.07	1.33	0.136	0.00%
B_28_YoY	-1.000	-0.927	-0.742	0.133	5.33	23.82	2,151.49	1.74	13.69%
D_48_Ave_12	-0.009	0.006	0.021	0.283	0.910	0.976	1.94	0.371	6.71%
D_48_Ave_3	-0.010	0.003	0.016	0.292	0.946	1.02	1.97	0.393	9.32%

One-hot encoding: 11 categorical fields checked

result · 0 columns created

table_05 · Categorical fields in the sample

Field	In sample	Cols	Field	In sample	Cols
B_30	No	0	D_114	No	0
B_38	No	0	D_116	No	0
D_63	No	0	D_117	No	0
D_64	No	0	D_120	No	0
D_66	No	0	D_126	No	0
D_68	No	0			

RESULT

None of the 11 categorical fields were among the 17 raw fields we drew, so encoding added zero columns.

one_hot_encode.py

```
# the 11 categorical fields from the brief
categorical = ["B_30", "B_38", ...]
present = [c for c in categorical
           if c in frame.columns]
encoded = pd.get_dummies(frame[present])
# nothing present, so 0 columns are added
```

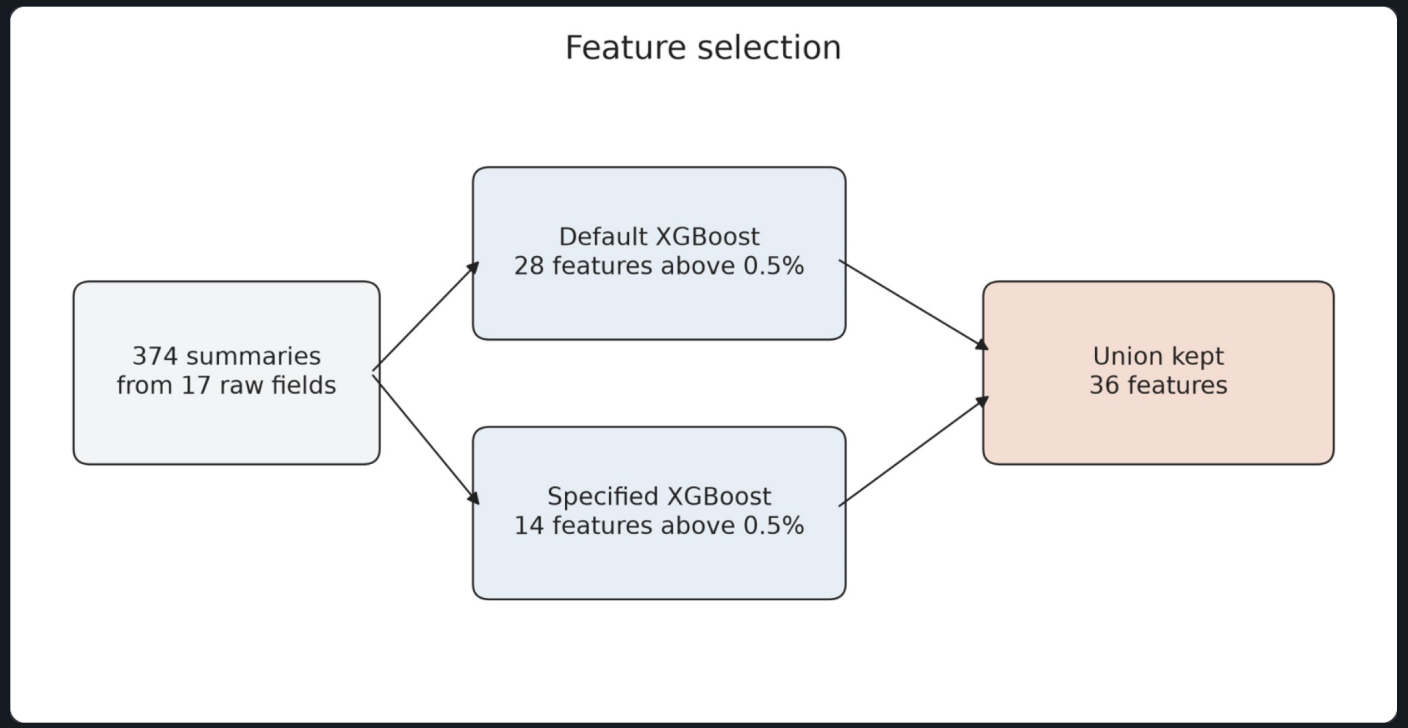
HOW WE TREATED THEM

We checked each named field against the sample and would have turned every category into its own 0/1 column. All 17 fields we kept are numeric.

Feature selection: 374 summaries to 36

attach · 06_feature_importance.xlsx

fig_06 · Selection process



A feature is kept when either model gives it more than 0.5%

table_06 · Features by category

Category	Total	Selected
D_*	220	23
S_*	22	1
P_*	0	0
B_*	88	8
R_*	44	4
Total	374	36

ATTACHED FILE

06_feature_importance.xlsx · one sheet per model + the 36 kept

XGBoost · Grid search: 72 models

saved after every model

grid_search_xgb.py

```
# 3x2x2x2x3 = 72 models, fit on train only
for n_trees in (50, 100, 300):
  for rate in (0.01, 0.1):
    for rows in (0.5, 0.8):
      for cols in (0.5, 1.0):
        for weight in (1, 5, 10):
          model = XGBClassifier(...)
          model.fit(X_train, y_train)
          # AUC: train, Test 1, Test 2
```

LESSONS LEARNED

A learning rate of 0.10 lifts train AUC faster than Test 2. Default weights of 5 and 10 did not win. **Winner: 50 trees, rate 0.10, 50% of accounts, all features, weight 1.**

settings.md · Why each dial is in the search

TREES · 50, 100, 300
Does extra fit still help on Test 2, or only on train?

LEARNING RATE · 0.01, 0.10
Small steps are stable; big steps can inflate train AUC.

ACCOUNTS PER TREE · 50%, 80%
Different slices keep one odd account from setting every split.

FEATURES PER TREE · 50%, 100%
Half draws make neighboring trees use different features.

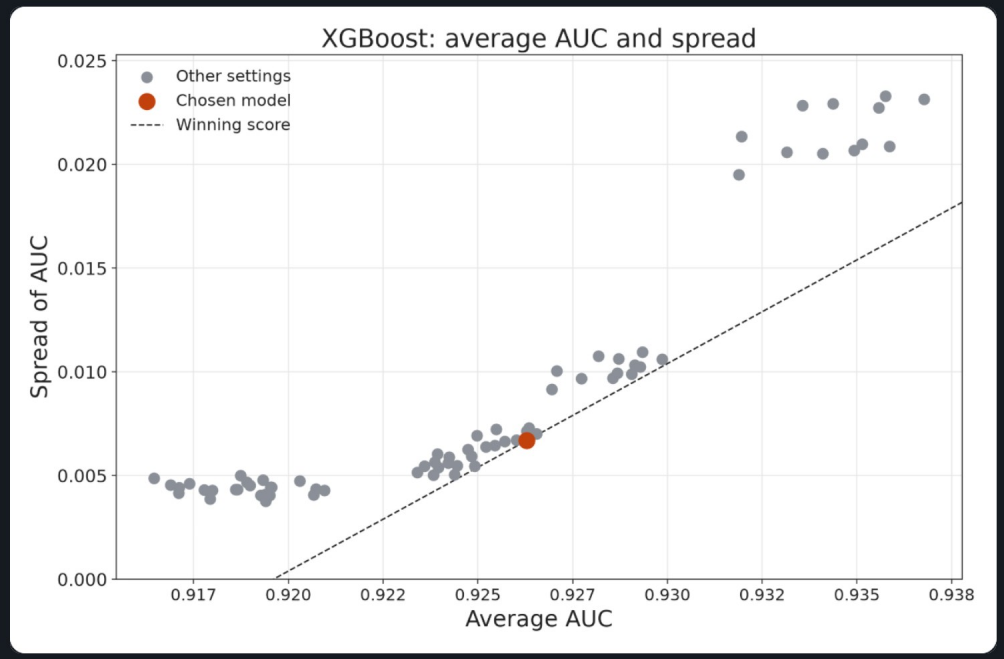
WEIGHT OF A DEFAULT · 1, 5, 10
Defaults are 25.65% of the sample; more weight catches more.

fixed · tree depth at XGBoost default · seed 42

XGBoost · Model selection

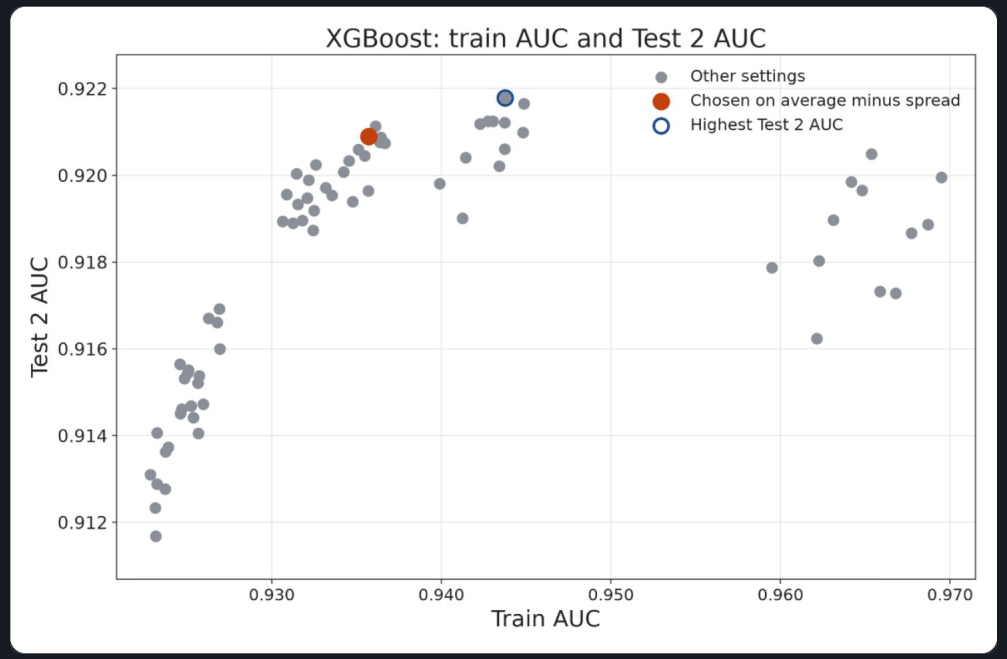
score · average AUC minus spread

fig_08a · Average AUC vs spread



PICK Orange: 50 trees, rate 0.10, 50% accounts, 100% features, weight 1. Best average AUC minus spread.

fig_08b · Train vs Test 2 AUC



PICK Open blue has the top Test 2 AUC (0.922) but a bigger train gap (0.944 train). We keep orange.

XGBoost · Final model

saved · xgb_final_model.json

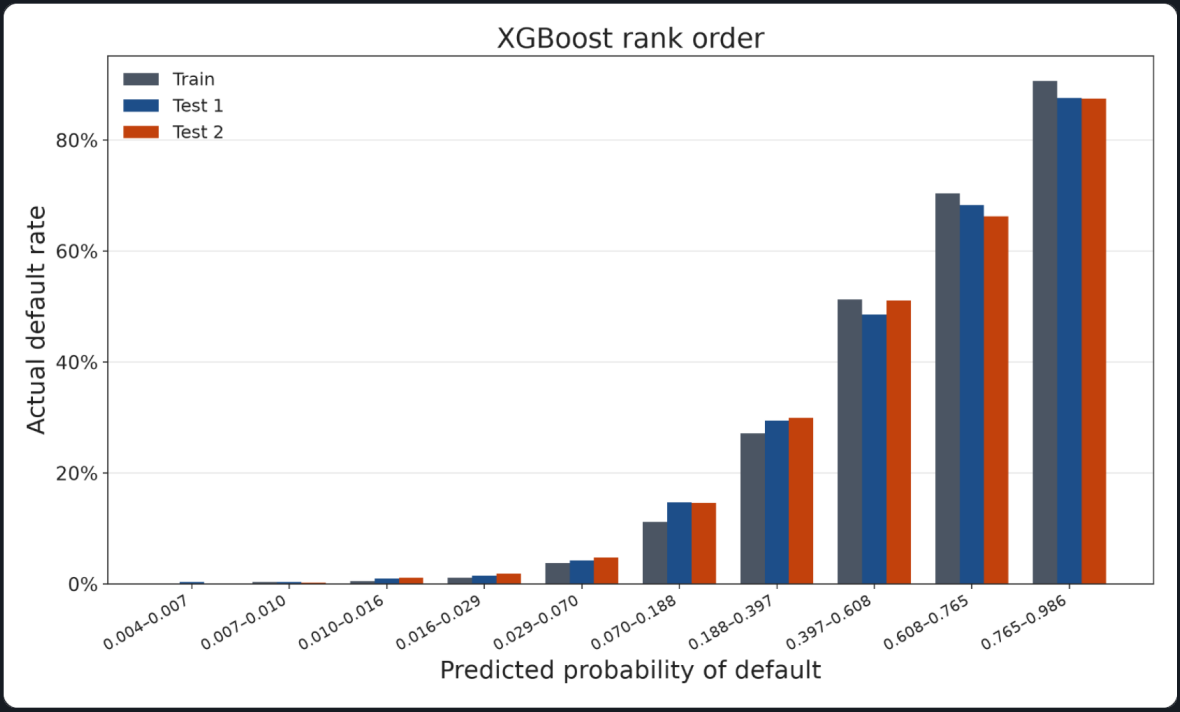
table_09 · Final parameters

Parameter	Value
Trees	50
Learning rate	0.10
Accounts per tree	50%
Features per tree	100%
Weight of a default	1

Train AUC	Test 1 AUC	Test 2 AUC
0.936	0.922	0.921

Avg 0.926 · Spread 0.007 · Score 0.920

fig_09 · Rank ordering by score bin

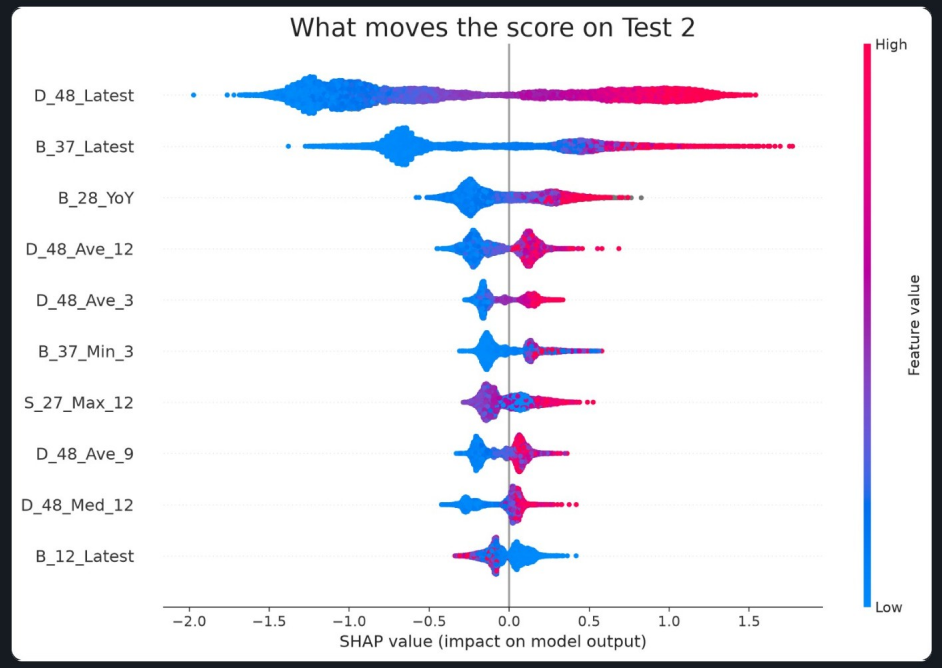


Bins cut on Train scores, same cuts on both tests. Default rate climbs from under 0.4% to 87.5%–90.6%.

XGBoost · SHAP beeswarm

sample · Test 2 · 13,768 accounts

fig_10 · SHAP values, top features



Right of zero pushes an account toward default. Red is a high feature value, blue a low one.

table_10 · Top 5 features by mean |SHAP|

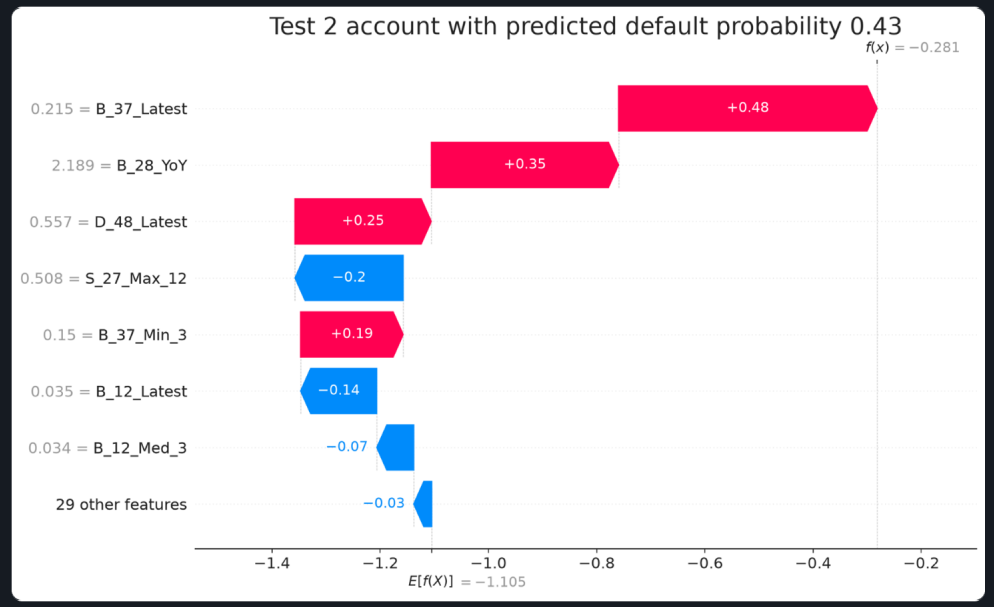
Rank	Attribute	What it is	Mean absolute SHAP	What the colors say
1	D_48_Latest	Latest delinquency	0.888	High value, red, sits on the right: more delinquency, higher default risk
2	B_37_Latest	Latest balance	0.576	Same direction: a larger balance raises the score
3	B_28_YoY	Balance change vs a year ago	0.234	A large increase raises the score
4	D_48_Ave_12	12-month average delinquency	0.171	Same direction as the latest delinquency
5	D_48_Ave_3	3-month average delinquency	0.145	Same direction, smaller effect

>_ 11 · XGBOOST

XGBoost · SHAP waterfall

sample · Test 2 · one account

fig_11 · What drives one account's score



Raw score, not probability. Starts at −1.105, the average account (about the 25.65% base rate). Red adds, blue subtracts. It ends at −0.281, a probability of 0.43.

account.json · PD 0.430 · did not default

Attribute	This account's value	Push	Compared with Test 2
B_37_Latest	0.215	+0.48 toward default	Median is 0.033, so this balance is high
B_28_YoY	2.19	+0.35 toward default	Median is 0.133, so the balance rose a lot
D_48_Latest	0.557	+0.25 toward default	Median is 0.310, so delinquency is above typical
S_27_Max_12	0.508	−0.20 away from default	Peak spend is the main attribute helping this account

THE BORDERLINE CASE

Rejected at 0.32, accepted at 0.54, and it paid: the exact band the strategy slide decides.

Neural network · Data processing

fit on train only

01
Start
The same 36 features. No new one-hot columns.

02
Cap and floor
Train 1st and 99th percentiles. Blanks set no fence.

03
Standardize
Subtract the train mean, divide by the train spread. Blanks stay blank.

04
Fill
Blanks become 0, the train average after scaling.

table_12 · Fences and scale learned on train

Feature	Missing	P1	P99	Mean	Scale
B_37_Latest	0.00%	0.000	1.08	0.136	0.223
D_110_Sum_12	99.11%	0.173	12.07	5.69	3.63
S_27_Max_12	14.82%	0.003	3.46	0.469	0.467

why_this_order.md

CAP BEFORE FILL

D_110_Sum_12 is blank on 99.11% of train accounts, yet its observed 1st and 99th percentiles still give the cap a range. Filling with 0 first would have crushed it.

>_ 13 · NEURAL NETWORK

Neural network · Grid search

32 models · 2×2×2×2×2

table_13 · What the search varies

Setting	Values
Hidden layers	2, 4
Nodes per layer	4, 6
Activation	ReLU, tanh
Dropout	50%, none
Batch size	100, 10,000

Fixed: Adam · 20 epochs · cross-entropy

nn_grid.py

```
for layers in (2, 4):
  for nodes in (4, 6):
    for act in (relu, tanh):
      for drop in (50, 100):
        for batch in (100, 10000):
          fit_and_score_auc(...)
```

lessons.md

BATCH SIZE

Batch 100 beat batch 10,000: mean Test 2 AUC 0.895 against 0.853.

DROPOUT

Keeping every node beat dropping half: 0.901 against 0.847.

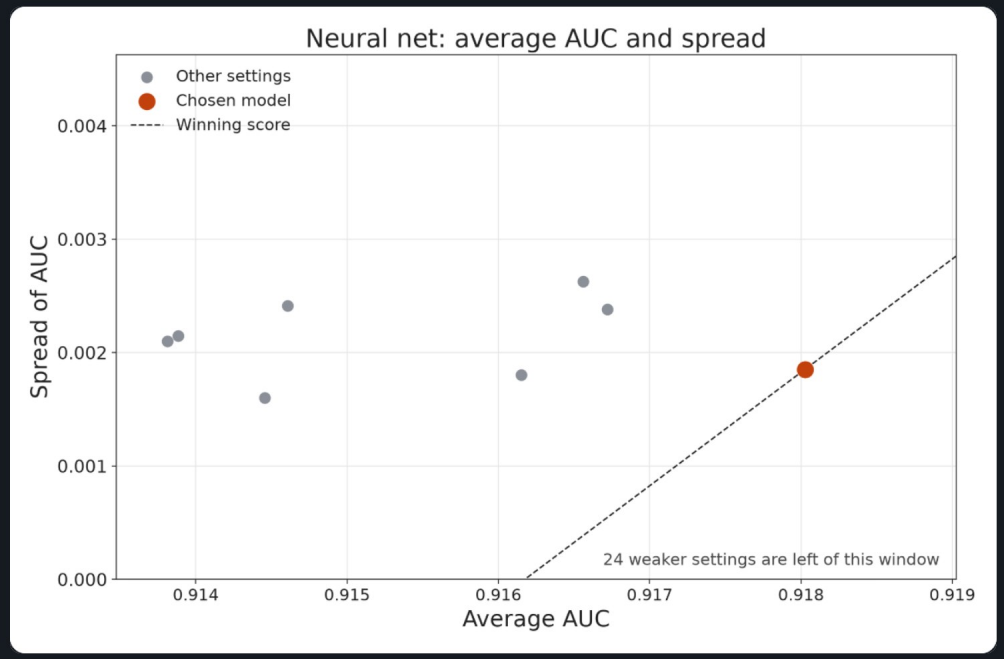
WINNER

2 layers, 6 ReLU nodes, no dropout, batch 100. One setting fell below 0.50.

Neural network · Model selection

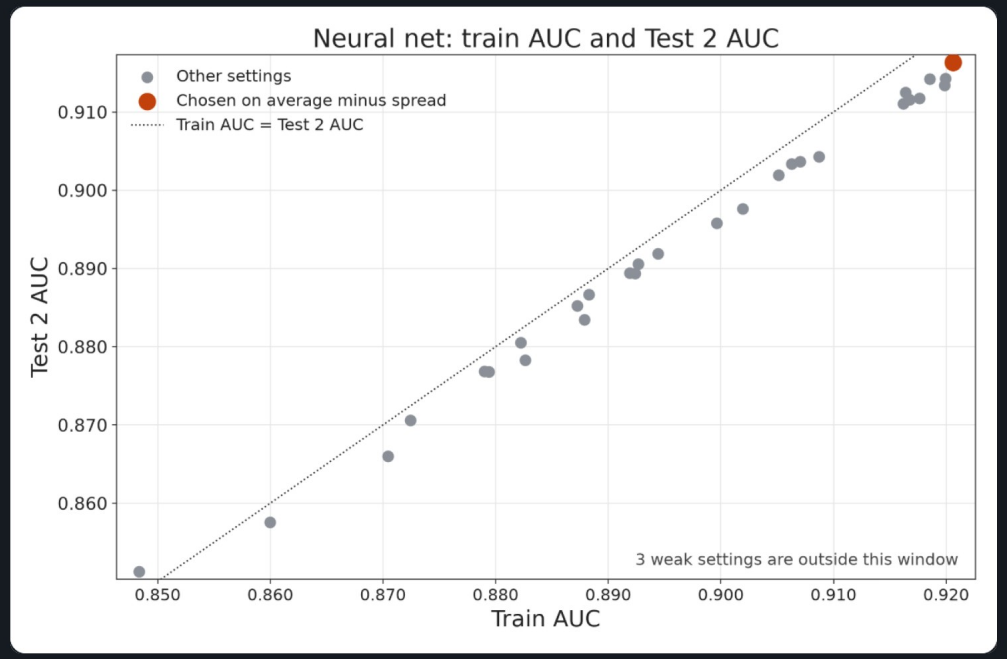
score · average AUC minus spread

fig_14a · Average AUC vs spread



PICK Orange is the only point on the winning line: **2 layers, 6 nodes, ReLU, no dropout, batch 100.**

fig_14b · Train vs Test 2 AUC



PICK The same point also has the top Test 2 AUC, 0.916. Both rules agree.

>_ 15 · NEURAL NETWORK

Neural network · Final model

saved · nn_final_model.keras

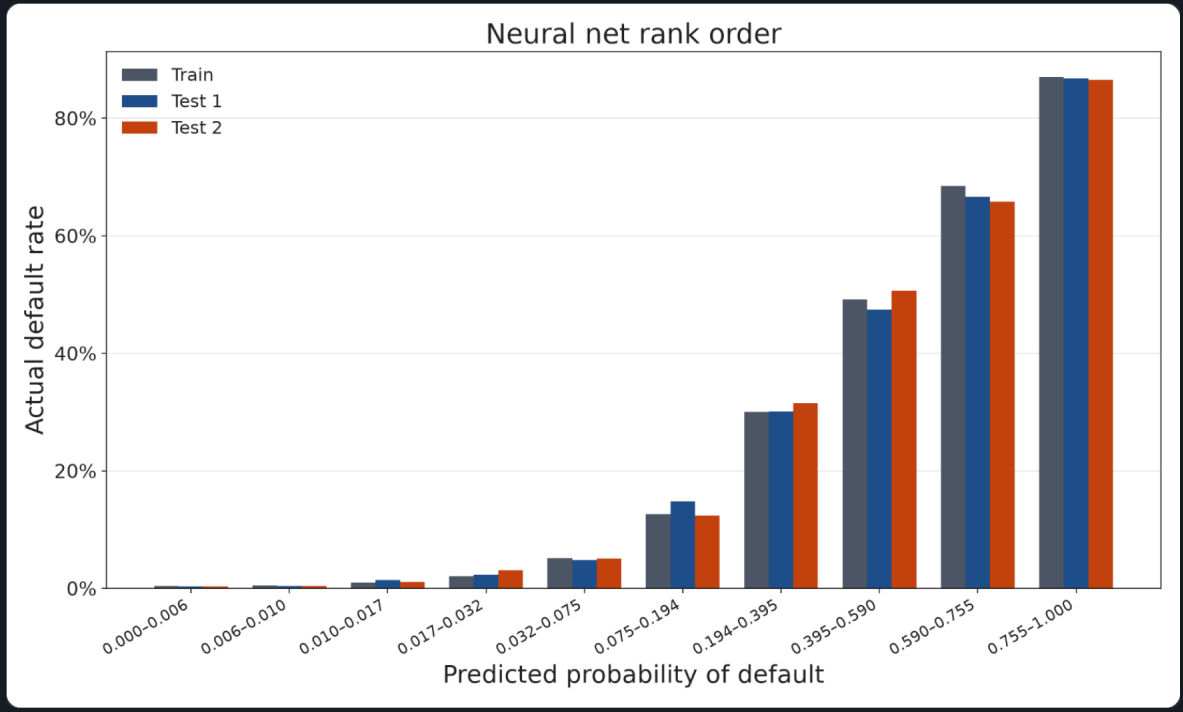
table_15 · Final parameters

Parameter	Value
Hidden layers	2
Nodes per layer	6
Activation	ReLU
Dropout	None
Batch size	100
Epochs	20
Optimizer	Adam
Loss	Cross-entropy

Train AUC	Test 1 AUC	Test 2 AUC
0.921	0.917	0.916

Avg 0.918 · Spread 0.002 · Score 0.916

fig_15 · Rank ordering by score bin



Bins cut on Train scores, same cuts on both tests. Default rate climbs from under 0.5% to 86.5%–87.0%.

>_ 16 · DECISION

Final model

same rule picks both winners

table_16 · AUC by sample

Model	Train	Test 1	Test 2	Average	Spread	Score	Chosen
XGBoost	0.936	0.922	0.921	0.926	0.007	0.920	Yes
Neural net	0.921	0.917	0.916	0.918	0.002	0.916	No

the_rule.md

SCORE

Average of the three AUCs, minus their spread. The same rule picks the winner inside each grid and between the finalists.

TIGHTER MODEL

The neural net is steadier: spread 0.002 against 0.007.
AUC uses probabilities assigned to count how often target probability (default) is higher than non-default.

VERDICT

XGBoost wins

Its average is higher by 0.008, more than the extra spread costs.

Score 0.920 against 0.916. Both rank-order well.

>_ 17 · DECISION

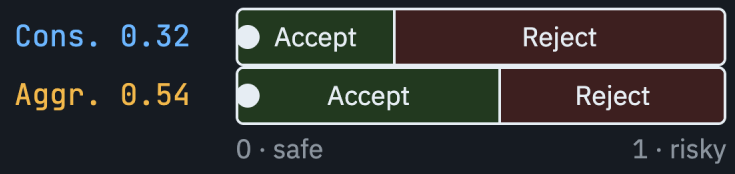
Strategy

accept if probability < threshold

table_17 · Default rate (DR) and 12-month revenue (feature u

Thr.	Train DR	Train rev.	Test 1 DR	Test 1 rev.	Test 2 DR	Test 2 rev.
0.10	1.54%	308.78	2.12%	67.01	2.16%	66.46
0.20	3.02%	418.47	3.99%	90.03	4.06%	90.53
0.30	4.52%	521.58	5.61%	110.38	5.64%	110.23
0.32	4.87%	541.72	5.80%	115.12	5.95%	114.28
0.40	6.38%	627.83	7.51%	131.80	7.69%	132.91
0.50	8.79%	727.77	9.89%	155.32	10.00%	154.05
0.54	9.86%	766.53	10.90%	162.03	11.00%	161.14
0.60	11.67%	824.01	12.19%	175.19	12.84%	173.73
0.70	15.35%	921.50	15.66%	194.95	15.84%	197.32
0.80	19.98%	1,008.92	19.99%	214.98	20.20%	217.70

fig_17 · The two strategy cutoff



RECOMMEND · 0.32

0.32 accepts the smaller book: 5.80% and 5.95% default on the tests. 0.54 hits 10.90% and 11.00%, over the 10% limit.

how_we_calculate.md

Default rate

$$= \frac{\text{accepted accounts that defaulted}}{\text{accepted accounts}}$$

```
accepted = data[data[pd_column] < threshold]
default_rate = accepted[target_column].mean()
```

Monthly revenue

= 0.02 × balance + 0.001 × spend

```
monthly = 0.02 * balance + 0.001 * spend
```

Account revenue

$$= \begin{cases} 12 \times \text{monthly revenue} & \text{if the account paid} \\ 0 & \text{if the account defaulted} \end{cases}$$

Total revenue

= add the account revenue of every accepted account

```
revenue = (12 * monthly).where(target == 0, 0).sum()
```